Project 2: Market-Basket Analysis on IMDb Movie Actors

JINGRU ZHAO

15/06/2026

1 Introduction

Market-basket analysis is commonly used to discover groups of items that frequently occur together in transactions. In a retail setting, a transaction is a customer's basket and the items are products. In this project, the same idea is applied to movies: each movie is considered a transaction, and the actors appearing in that movie are the items.

The goal of this project is to identify frequent actor combinations and association rules among actors. In practical terms, the analysis answers questions such as:

- Which actors appear most frequently in the IMDb Top 1000 dataset?
- Which pairs or groups of actors often appear in the same movie?
- If one actor appears in a movie, how likely is another actor, or group of actors, to appear as well?

The main technique used is the Apriori algorithm, which is appropriate for scalable frequent itemset mining.

2 Dataset

The selected dataset is the *IMDb Movies Dataset*, available on Kaggle at:

https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows

The dataset is released under the Public Domain license. It contains information about 1000 highly rated movies and TV shows, including title, year, genre, director, IMDb rating, and four main actors.

For this project, only the actor columns are used:

- `Star1`
- `Star2`
- `Star3`
- `Star4`

Other columns, such as title, genre, rating, and gross revenue, are not used as input to the market-basket analysis.

3 Data Organization

In the original CSV file, each row describes one movie. this project study actor co-occurrence, I did not treat the movie itself as the item to be analyzed. Instead, each movie is represented as one basket containing its main actors.

For each movie, I extracted the four columns `Star1`, `Star2`, `Star3`, and `Star4`. Together, these four actor names form the basket associated with that movie. For example, if a movie contains the following actors:

```
Star1 = "Daniel␣Radcliffe"
Star2 = "Emma␣Watson"
Star3 = "Rupert␣Grint"
Star4 = "Michael␣Gambon"
```

then the corresponding basket is:

```
{
    "Daniel␣Radcliffe",
    "Emma␣Watson",
    "Rupert␣Grint",
    "Michael␣Gambon"
}
```

In this representation, the whole set of actor names extracted from one movie is one basket, while each individual actor name inside that set is one item. Therefore, if two or more actors appear in the same basket, it means that they appeared together in the same movie. A group of actors, such as {`"Daniel Radcliffe", "Emma Watson"`}, is then treated as an itemset whose frequency can be measured across all movies in the dataset.

4 Data Download and Replicability

The dataset is downloaded using the Kaggle API. The code uses the Kaggle dataset identifier:

```
harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows
```

The project avoid exposing sensitive Kaggle credentials. In the public version of the code, the credential placeholders are kept as:

```
KAGGLE_USERNAME = "xxxxxx"
KAGGLE_KEY = "xxxxxx"
```

5 Pre-processing

The pre-processing stage transforms the original CSV data into a list of clean transactions. The following steps are applied:

1. Only the columns `Star1`, `Star2`, `Star3`, and `Star4` are selected.
2. Missing actor names are ignored.
3. Leading and trailing spaces are removed.
4. Empty strings are discarded.
5. Actor names are normalized using Unicode normalization to reduce formatting differences.
6. Non-breaking spaces and repeated spaces are replaced with standard single spaces.
7. Each movie basket is stored as a Python set, so duplicate actor names within the same movie are removed.

These pre-processing steps are sufficient for the selected dataset because the actor fields are already relatively clean. More complex entity resolution, such as detecting real-world aliases or disambiguating actors with the same name, is outside the scope of this project.

6 Methodology

6.1 Frequent Itemsets

An itemset is considered frequent if its support is greater than or equal to a selected threshold. The support count of an itemset is the number of movies in which all actors in that itemset appear together. The support is the support count divided by the total number of movies:

$$\text{support}(X) = \frac{\text{support\_count}(X)}{N}$$

where X is an itemset and N is the total number of transactions.

6.2 Association Rules

Association rules have the form:

$$A \to B$$

where A is the antecedent and B is the consequent. In this project, both A and B are actors or actor groups.

The confidence of a rule measures how often the consequent appears when the antecedent appears:

$$\text{confidence}(A \to B) = \frac{\text{support\_count}(A \cup B)}{\text{support\_count}(A)}$$

The lift of a rule measures whether the association is stronger than random co-occurrence:

$$\text{lift}(A \to B) = \frac{\text{confidence}(A \to B)}{\text{support}(B)}$$

A lift greater than 1 indicates a positive association.

6.3 Apriori Algorithm

The Apriori algorithm is used to discover frequent itemsets. It relies on the downward-closure property: if an itemset is frequent, all of its subsets must also be frequent. Conversely, if an itemset is not frequent, none of its supersets can be frequent.

The implemented procedure is:

1. Count all frequent single actors.
2. Generate candidate actor pairs from frequent single actors.
3. Count candidate pairs and keep only those satisfying the support threshold.
4. Repeat the process for itemsets of size 3 and 4.
5. Generate association rules from the discovered frequent itemsets.

Since the dataset contains exactly four actor columns per movie, the maximum itemset size is set to 4.

7 Scalability

The Apriori algorithm expands candidates level by level and prunes candidates whose subsets are not frequent.

The data structure used for each basket is a Python set, which allows efficient membership checks. Candidate counting is performed by scanning the transactions and generating only the combinations that can appear within each movie basket. In this dataset, each transaction contains at most four actors, so the number of combinations per basket is naturally bounded.

For larger datasets, the same implementation can still process more rows because it scans transactions iteratively. However, for datasets with much larger baskets, the number of candidate combinations may grow quickly such as the full cast list instead of only four main actors, the number of candidate itemsets could grow quickly.. In that case, further optimizations such as hash-based FP-Growth, or distributed processing would be appropriate.

The code also includes a global subsampling option. This allows experiments to be run on a smaller subset during development while keeping the same logic for full-data execution.

8 Experimental Setup

The experiment was carried out as a market-basket analysis task. In this setting, each movie is treated as one basket, and the actors in `Star1`, `Star2`, `Star3`, and `Star4` are treated as the items inside that basket. The aim is to find which actors often appear together in the same movies and to produce association rules between actors.

The full dataset of 1000 movies was used in the final experiment. The process was the same for each run: first, the dataset was downloaded through the Kaggle API; then, the four actor columns were extracted and cleaned; after that, each movie was converted into a basket of actors. Finally, the Apriori algorithm was applied to find frequent actor groups, and association rules were generated from these groups.

Because the dataset contains 2709 unique actors but only four actor fields for each movie, repeated actor combinations are not very common. For this reason, different support and confidence values were tried before choosing the final setting.

Two settings were considered:

- **Loose setting:** `min_support` $= 0.002$, `min_confidence` $= 0.20$. This means that an actor group only needs to appear in at least two movies, so it gives more results.
- **Final setting:** `min_support` $= 0.003$, `min_confidence` $= 0.50$. This means that an actor group must appear in at least three movies, and an association rule is kept only if its confidence is at least 50%.

The loose setting produced more frequent itemsets and association rules, but many of them were based on only two co-appearances. These results were less reliable, because a very high confidence or lift value can be caused by a small number of repeated movies. For this reason, the final setting was chosen. It keeps a reasonable number of results while removing many weak patterns based on only two co-appearances.

The selected parameters for the final run were:

Parameter	Value
Minimum support	0.003
Minimum support count	3 movies
Minimum confidence	0.50
Maximum itemset size	4.00
Number of baskets	1000
Number of unique actors	2709
Average basket size	4.00

Table 1: Final experimental configuration.

The confidence threshold was set to 0.50 so that the consequent must appear in at least half of the cases where the antecedent appears. This filters weak rules while keeping interpretable actor associations.

9 Results

9.1 Frequent Itemsets

The final run produced 299 frequent itemsets. Their distribution by size is shown in Table 3.

Parameter	Value
Minimum support	0.003
Minimum support count	3 movies
Minimum confidence	0.50
Maximum itemset size	4
Number of baskets	1000
Number of unique actors	2709
Average basket size	4.00

Table 2: Final experimental configuration.

The confidence threshold was set to 0.50 so that the consequent must appear in at least half of the cases where the antecedent appears. This filters weak rules while keeping interpretable actor associations.

10 Results

10.1 Frequent Itemsets

The final run produced 299 frequent itemsets. Their distribution by size is shown in Table 3.

Itemset size	Number of frequent itemsets
1	271
2	25
3	3
4	0

Table 3: Number of frequent itemsets by size.

Most frequent itemsets have size 1, which means that single actors appear more often than specific actor groups. However, the actor pairs and triples are more useful for this project because they show repeated co-appearances between actors.

The most frequent individual actors are shown in Table 4.

Actor	Support count	Support
Robert De Niro	17	0.017
Tom Hanks	14	0.014
Al Pacino	13	0.013
Brad Pitt	12	0.012
Clint Eastwood	12	0.012
Christian Bale	11	0.011
Leonardo DiCaprio	11	0.011
Matt Damon	11	0.011
James Stewart	10	0.010
Denzel Washington	9	0.009

Table 4: Top frequent single actors.

The most frequent actor is Robert De Niro, with 17 appearances out of 1000 movies. This shows that the dataset is sparse: even frequent actors appear in only a small part of the dataset.

10.2 Association Rules

The final run produced 46 association rules. Some examples are shown in Table 5.

Antecedent	Consequent	Support count	Confidence	Lift
Elijah Wood	Ian McKellen, Orlando Bloom	3	1.00	333.33
Mark Hamill	Carrie Fisher, Harrison Ford	3	1.00	333.33
Daniel Radcliffe	Rupert Grint	6	1.00	166.67
Rupert Grint	Daniel Radcliffe	6	1.00	166.67
Daniel Radcliffe, Emma Watson	Rupert Grint	5	1.00	166.67
Emma Watson, Rupert Grint	Daniel Radcliffe	5	1.00	166.67
John Turturro	Ethan Coen	3	1.00	166.67
Elijah Wood	Ian McKellen	3	1.00	142.86

Table 5: Examples of association rules.

Several strong rules are related to well-known movie series. For example, Daniel Radcliffe, Emma Watson, and Rupert Grint are associated with *Harry Potter*; Carrie Fisher, Harrison Ford, and Mark Hamill are associated with *Star Wars*; and Elijah Wood, Ian McKellen, and Orlando Bloom are associated with *The Lord of the Rings*.

The rule `Daniel Radcliffe` $\rightarrow$ `Rupert Grint` has support count 6 and confidence 1.00. This means that, in this dataset, all baskets containing Daniel Radcliffe also contain Rupert Grint. The high lift values show that these co-occurrences are stronger than random co-occurrence within the dataset. show that these co-occurrences are stronger than random co-occurrence within the dataset.

11 Discussion

The results show that market-basket analysis can help find actors who often appear together in the same movies. However, the results need to be interpreted carefully. The dataset has 1000 movies and 2709 unique actors, but each movie only gives four actor names. Because of this, most actor combinations do not appear many times.

This is why the choice of support threshold matters. When `min_support` = 0.002, actor groups that appear only twice are also kept. This gives more results, but some of them are not very strong because they are based on only two movies. In the final version, I used `min_support` = 0.003, so an actor group must appear at least three times. This gives fewer results, but they are more useful to discuss.

The association rules also need some caution. For example, a confidence value of 1.00 means that, in this dataset, whenever the first actor or actor group appears, the second one also appears. But if the support count is only 3, the rule is still based on just three movies. For this reason, I did not look at confidence alone, but also considered support count and lift.

Some of the strongest results come from movie series, such as *Harry Potter*, *Star Wars*, and *The Lord of the Rings*. This is expected, because movie series often use the same actors across several films. Therefore, the results are best understood as repeated co-appearances inside this IMDb dataset, rather than general rules about all movies or all actors.

12 Limitations

The main limitations of the project are:

- The dataset contains only 1000 movies, so the number of transactions is limited.
- Each movie includes only four actor fields, which may omit other relevant actors.
- The analysis does not use movie year, genre, director, or rating.
- Some discovered associations are driven by movie franchises, which may dominate the results.
- Actor name normalization handles simple formatting differences but not complex real-world identity resolution.

13 Conclusion

This project implemented a market-basket analysis system using the IMDb Movies Dataset. Movies were represented as baskets and actors were represented as items. After pre-processing, the Apriori algorithm was used to find frequent actor itemsets and association rules.

The final experiment used a minimum support of 0.003 and a minimum confidence of 0.50. The analysis found 299 frequent itemsets and 46 association rules. The most interpretable results correspond to repeated actor groups from well-known film series, such as *Harry Potter*, *Star Wars*, and *The Lord of the Rings*. The results demonstrate that market-basket analysis can reveal meaningful co-occurrence patterns in movie actor data, while also showing the importance of careful threshold selection in sparse datasets.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism,

collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.“